

Assignment Overview

Students are required to choose one topic from the list, implement it, and create a report and presentation slides.

Report Content

1. **Introduction:**
 - Provide a general overview of the topic.
 - Clearly state the problem that the project aims to solve.
2. **Main Content:**
 - Describe the chosen algorithms or solutions for the problem.
 - Describe the program:
 - Input/output data.
 - Functions and their purposes.
3. **Program Interface:**
 - (Optional) Describe the program's graphical user interface.
4. **Teamwork:**
 - Clearly outline each team member's contributions.
5. **References:**
 - List all references used. Failure to do so will result in a zero score.

Presentation Slides

- Create slides based on the report content.
- Presentation time: 15 minutes.
- Avoid directly copying text from the report.
- All team members must understand the project.

List of Topics

1. Process communication IPC in Windows: Shared Memory, Message Queues, RPC

Implement file map view in windows, pipes in Linux, a simple RPC (Remote Procedure Calls) system using a language of your choice.
Discuss the advantages and disadvantages of IPC mechanisms.
Evaluate the performance of your IPC system.

2. Process communication IPC in Linux: Pipe, Message Queues, RPC

Implement file map view in windows, pipes in Linux, a simple RPC (Remote Procedure Calls) system using a language of your choice.
Discuss the advantages and disadvantages of IPC mechanisms.
Evaluate the performance of your IPC system.

3. GPU-Aware Task Scheduler with Memory Optimization

Study GPU scheduling basics, Memory sharing CPU–GPU

Simulate task scheduling policies

Benchmark/evaluate

4. Multi-Level Feedback Queue Scheduling

Implement a MLFQ scheduler and evaluate its performance.

Compare MLFQ to other scheduling algorithms.

Discuss the factors that affect the performance of MLFQ.

5. Scheduling Algorithms in Multiprocessor Systems

Implement and evaluate different scheduling algorithms for multiprocessor systems.

Discuss the challenges of scheduling in multiprocessor systems.

Compare the performance of different algorithms.

6. Reader-Writer problem

7. Dining Philosophers problem

8. Barber shop problem

9. Barrier synchronization

10. Producer and Consumer problem

6-10. Synchronization Problems

Implement solutions for the specified synchronization problems.

Analyze the correctness and efficiency of your solutions.

Compare your solutions to other known solutions.

11. Monitor-Based Synchronization in Java/Python

Implement a monitor class in Java/Python and use it to solve synchronization problems.

Discuss the benefits of using monitors for synchronization.

Compare monitors to other synchronization mechanisms.

12. Online Ticket Selling System

Implement a distributed system for online ticket sales.

Ensure that the system is scalable and fault-tolerant.

Evaluate the performance of the system.

13. Process Management in Linux

Implement a simple process manager in Linux (using Linux API function).

Demonstrate your understanding of process creation, termination, and scheduling.

Discuss the challenges of process management.

14-16. Memory Management

Implement paging, segmentation, or virtual memory.

Evaluate the performance of different memory management techniques.

Discuss the trade-offs between different techniques.

17. Ext4 File System

Implement a simple file system based on Ext4.

Demonstrate your understanding of file system concepts.

Evaluate the performance of your file system.

18. Btrfs/ZFS File System

Implement a simple file system based on Btrfs/ZFS.

Demonstrate your understanding of file system concepts.

Evaluate the performance of your file system.

19. Animation

Create a creative and informative animation about an OS topic.

Clearly explain the concepts in a visually appealing way.

Ensure that the animation is well-produced and professional.

II. AI-related topics that can be explored in the context of operating systems:

20. AI Task Scheduling:

Implement an intelligent task scheduler that uses machine learning algorithms to predict the resource needs of different tasks and optimally schedules them on the system.

21. Resource Allocation with Machine Learning:

Develop a system that uses AI techniques to dynamically allocate system resources (CPU, memory, etc.) based on the requirements and usage patterns of running applications.

22. Autonomous System Management:

Create an autonomous system manager that uses AI to monitor the health of the system, predict potential issues, and take proactive measures to prevent or mitigate problems.

23. Self-Optimizing Systems:

Design an operating system that can autonomously optimize its configurations and parameters using reinforcement learning or other AI methods to improve performance and energy efficiency.

24. Intelligent Memory Management:

Utilize machine learning to predict and adaptively manage memory allocation based on application behavior and usage patterns, optimizing for both performance and resource efficiency.

25. AI-Based Security Enhancements:
Implement AI-driven intrusion detection and prevention mechanisms within the operating system to identify and respond to security threats in real-time.
26. Predictive File System Caching:
Develop a file system that employs machine learning models to predict which files are likely to be accessed in the near future and proactively caches them for improved I/O performance.
27. Smart Power Management:
Use AI algorithms to analyze usage patterns and predict periods of low activity, allowing the operating system to intelligently adjust power settings to conserve energy without sacrificing performance.
28. Personalized User Experience:
Create an operating system interface that learns and adapts to user preferences over time using AI, providing a more personalized and efficient user experience.
29. Distributed AI Processing:
Explore how operating systems can facilitate the deployment and management of distributed AI applications, ensuring efficient communication and resource utilization across a network of machines.

III.Extended research topics

30. System Call Implementation:
Details: Create a set of custom system calls that perform unique operations not covered by standard system calls. Examples could include interacting with specialized hardware or implementing advanced file operations.
Hints: Understand the anatomy of system calls, the role of the syscall interface, and how parameters are passed between user and kernel space.
31. Kernel Module Development:
Details: Develop kernel modules that extend the functionality of the operating system dynamically. This could involve adding support for new devices, filesystems, or security features.
Hints: Familiarize yourself with the Linux kernel module framework or the equivalent in the operating system of your choice. Understand how to load, unload, and interact with kernel modules.
32. Concurrency Control in the Kernel:
Details: Implement synchronization mechanisms within the kernel to manage concurrent access to shared resources. This could include designing and implementing locks, semaphores, or other synchronization primitives.
Hints: Study the principles of concurrent programming, understand race conditions and critical sections, and explore how these concepts are applied at the kernel level.

33. Real-time Operating System (RTOS):
Details: Develop or enhance an operating system to support real-time requirements, ensuring timely and predictable responses to external events.
Hints: Learn about real-time scheduling algorithms, priority inversion, and techniques for minimizing latency. Consider how to guarantee deadlines for critical tasks.
34. Process and Thread Management:
Details: Extend the process and thread management capabilities in the kernel. This might involve implementing features like lightweight processes, thread pooling, or user-level threads.
Hints: Dive into the internals of process and thread management, understand context switching, and explore ways to optimize the creation and destruction of processes and threads.
35. Interrupt Handling Optimization:
Details: Improve the efficiency of interrupt handling routines in the kernel, optimizing response times and minimizing disruptions to the system.
Hints: Study interrupt handling mechanisms in the kernel, explore ways to reduce interrupt latency, and consider optimizations such as interrupt coalescing.
36. Memory Allocator Design:
Details: Create a custom memory allocator for the operating system, optimizing for speed, fragmentation, and scalability.
Hints: Understand different memory allocation algorithms (e.g., buddy allocation, slab allocation), study existing allocators in operating systems, and experiment with designing your own.
37. Containerization in the Kernel:
Details: Explore the integration of containerization technologies at the kernel level, allowing for lightweight and isolated application environments.
Hints: Study containerization principles, understand container runtimes, and explore how namespaces and cgroups can be managed at the kernel level.
38. Dynamic Linker/Loader Implementation:
Details: Implement a dynamic linker/loader for the operating system, enabling dynamic linking of shared libraries at runtime.
Hints: Explore the ELF (Executable and Linkable Format) specification, understand how dynamic linking works, and consider implementing lazy loading and symbol resolution.
39. Fault Tolerance Mechanisms:
Details: Introduce fault tolerance mechanisms at the kernel level, such as checkpointing, process migration, or recovery strategies to enhance system reliability.
Hints: Study fault tolerance concepts, explore techniques like process checkpointing and migration, and consider how to recover from failures gracefully.